CHAPTER 3

FUNCTIONS

CHAPTER ESSENCE AT A GLANCE:
Functions decompose complex software systems into modular reusable procedural components, managing formal-actual parameter binding, variable scope, and stack frame lifecycles.

4 CORE PRINCIPLES TO UNDERSTAND FIRST

• **Modular Decomposition and Reusability:** Dividing complex monolithic programs into independent functional subprograms simplifies algorithmic verification, eliminates repetitive code, facilitates team development, and isolates maintenance bugs.

• **Function Prototyping and Signature Declaration:** A function prototype announces the subprogram interface to the compiler before invocation, specifying its return type, identifier, and parameter type list to enforce type safety.

• **Parameter Passing Semantics:** Call by Value copies actual argument values into isolated formal parameter stack locations, whereas Call by Reference passes aliases of caller memory locations using reference parameters to permit direct modification.

• **Scope Boundaries and Standard Library Modules:** Variables declared within a function block possess local scope and automatic storage duration, isolated from global identifiers, while standard libraries provide pre-tested mathematical and character evaluation utilities.

ESSENTIAL ARCHITECTURE & CORE SYNTAX CARDS

$$\text{Function Prototype Syntax} = \text{ReturnType FunctionName}(\text{Type}_1\ p_1,\ \text{Type}_2\ p_2,\ \ldots);$$

Subprogram Interface Signature

$$\text{Parameter Binding} = \frac{\text{Call by Value: Formal} \leftarrow \text{Isolated Copy of Actual Value}}{\text{Call by Reference: Formal} \equiv \&\text{Actual Variable (Direct Alias)}}$$

Parameter Passing Memory Paradigms

$$\text{Standard Math Library} = \text{sqrt}(x) = \sqrt{x},\quad \text{pow}(x,y) = x^y,\quad \text{abs}(x) = |x| \quad (\text{in } <\text{cmath}>)$$

Mathematical Built-In Functions

$$\text{Character Evaluation} = \text{isalpha}(c),\ \text{isdigit}(c),\ \text{toupper}(c),\ \text{tolower}(c) \quad (\text{in } <\text{cctype}>)$$

Character Classification Utilities

CRUCIAL EXAM PRECAUTIONS & VALUATION RUBRIC SECRETS

• **Alert:** Never omit the function prototype if the function definition is written below main(); omitting it produces an undeclared identifier compilation error.

• **Alert:** In Call by Value, modifications made to formal parameters inside the function body have absolutely zero effect on the actual arguments in the calling function.

• **Alert:** A non-void function must execute a return statement with a compatible return value along every possible execution path; omitting return causes undefined output.

HIGH-YIELD CENTUM QUESTIONS & STEP-BY-STEP EXPLANATIONS

1. What is a function prototype? Why is it mandatory when a function definition follows the main function in C++? [Score: 2 Marks]

Answer & Key Valuation Steps:

• Definition: A declaration statement that informs the compiler about the function name, return type, and number and types of its arguments before the function is called. **[1 Mark]**

• Necessity: Enables the compiler to check argument count and data type compatibility during compilation before encountering the full function definition. **[1 Mark]**

2. Distinguish between actual parameters and formal parameters in C++ with an illustrative example. [Score: 2 Marks]

Answer & Key Valuation Steps:

• Actual Parameters: The actual values or variables passed to a function inside the function call statement (e.g. In display(x, y); x and y are actual parameters). **[1 Mark]**

• Formal Parameters: The variables declared in the function header that receive the incoming values or references from the calling function (e.g. In void display(int a, int b); a and b are formal parameters). **[1 Mark]**

3. Compare Call by Value and Call by Reference parameter passing methods with memory illustrations and code syntax. [Score: 3 Marks]

Answer & Key Valuation Steps:

• Call by Value: A duplicate copy of the actual parameter is passed to the formal parameter. Changes inside the function do not affect the original caller variable. Uses standard syntax void func(int x). **[1.5 Marks]**

• Call by Reference: An alias (reference) of the actual parameter is passed using the ampersand operator (&). Changes inside the function directly modify the caller variable. Uses syntax void func(int &x). **[1.5 Marks]**

4. Explain four built-in character handling functions from the <cctype> header file: isupper(), islower(), isdigit(), and toupper(). [Score: 3 Marks]

Answer & Key Valuation Steps:

• isupper(c): Checks if character c is an uppercase letter (A-Z); returns non-zero (true) if true, 0 if false. **[0.75 Mark]**

• islower(c): Checks if character c is a lowercase letter (a-z); returns non-zero (true) if true, 0 if false. **[0.75 Mark]**

• isdigit(c): Checks if character c is a numerical decimal digit (0-9); returns non-zero if true, 0 if false. **[0.75 Mark]**

• toupper(c): Converts a lowercase character to its corresponding uppercase character; leaves non-lowercase characters unchanged. **[0.75 Mark]**

5. Write a C++ user-defined function named swapValues using Call by Reference to interchange the values of two integer variables. [Score: 3 Marks]

Answer & Key Valuation Steps:

• Function Header: void swapValues(int &a, int &b) { **[1 Mark]**

• Interchange Logic: int temp = a; a = b; b = temp; } **[1 Mark]**

• Explanation: Because reference parameters &a and &b alias the original caller variables, swapping temp directly alters the calling scope values without returning. **[1 Mark]**

6. Explain the difference between local variables and global variables regarding scope, lifetime, and default initial value. [Score: 3 Marks]

Answer & Key Valuation Steps:

• Scope: Local variables are accessible only within the specific block or function where declared; global variables are accessible by all functions throughout the file. **[1 Mark]**

• Lifetime: Local variables are created on block entry and destroyed on block exit; global variables persist in memory throughout the entire program run. **[1 Mark]**

• Default Initialization: Local variables contain indeterminate garbage values if uninitialized; global variables are automatically initialized to zero. **[1 Mark]**

7. Write a complete C++ modular program with separate user-defined functions to: (a) calculate factorial of a number, (b) compute power x raised to y, and invoke both from main(). [Score: 4 Marks]

Answer & Key Valuation Steps:

• Factorial Function: long factorial(int n) { long f = 1; for (int i = 1; i <= n; i++) f *= i; return f; } **[1 Mark]**

• Power Function: double power(double x, int y) { double res = 1.0; for (int i = 0; i < y; i++) res *= x; return res; } **[1 Mark]**

• Main Function Setup: Include iostream, using namespace std, define prototypes or place functions above main(), and prompt user for inputs. **[1 Mark]**

• Invocation and Display: Call factorial(5) and power(2.0, 3), display calculated results cleanly, and return 0. **[1 Mark]**